

R-Code Beispiele aus WDDA Vorlesung 03

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-04-29

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Lagemasse	2
3.1	Arithmetischer Mittelwert	3
3.2	Median	3
3.3	Modus	4
3.4	Gewichteter Mittelwert	6
4	Streuungs- und Streuungsmasse	7
4.1	Spannweite	7
4.2	Quartile und Interquartilsabstand (IQR)	9
4.3	Varianz und Standardabweichung	11
5	Empirische Regel	13
6	Standardisierung	15
7	Schiefheit	18
8	Zusammenhangsmasse	20
8.1	Kovarianz	20
8.2	Korrelationskoeffizient	23
9	Visualisierung	26
9.1	Boxplots	26

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der WDDA Lecture 03 Folien zusammen. Der Fokus liegt auf der deskriptiven Statistik, insbesondere auf numerischen Massen wie Lage-, Streuungs- und Zusammenhangsmasse.

Die deskriptive Statistik ist ein fundamentaler Bereich der Statistik, der sich mit der Zusammenfassung, Beschreibung und Darstellung von Daten befasst. Im Gegensatz zur inferentiellen Statistik, die Schlussfolgerungen über eine Grundgesamtheit auf Basis von Stichproben zieht, beschreibt die deskriptive Statistik die vorliegenden Daten ohne darüber hinausgehende Schlussfolgerungen.

Numerische Masse in der deskriptiven Statistik können in drei Hauptkategorien eingeteilt werden:

1. **Lagemasse:** Beschreiben das Zentrum oder die zentrale Tendenz der Daten (z.B. Mittelwert, Median)
2. **Streuungsmaße:** Beschreiben die Variabilität oder Dispersion der Daten (z.B. Varianz, Standardabweichung)
3. **Zusammenhangsmaße:** Beschreiben die Beziehung zwischen zwei oder mehr Variablen (z.B. Kovarianz, Korrelation)

2 Vorbereitung

```
# Pakete laden
library(readxl) # Paket zum Einlesen von Excel-Dateien

# Daten importieren
Graduates <- read_excel("../data/WDDA_03.xlsx", sheet = "Graduates")
Brand <- read_excel("../data/WDDA_03.xlsx", sheet = "Auto")
Advertising <- read_excel("../data/WDDA_03.xlsx", sheet = "Advertising")

# Variablen extrahieren für spätere Verwendung
Salary <- Graduates$Salary # Extrahiert die Spalte 'Salary' aus dem Graduates-Dataframe
Major <- Graduates$Major   # Extrahiert die Spalte 'Major' aus dem Graduates-Dataframe
sales <- Advertising$sales # Extrahiert die Spalte 'sales' aus dem Advertising-Dataframe
adverts <- Advertising$adverts # Extrahiert die Spalte 'adverts' aus dem
  ↪ Advertising-Dataframe

# Überprüfen der Datenstruktur
str(Graduates) # Zeigt die Struktur des Graduates-Dataframes

## tibble [60 x 2] (S3: tbl_df/tbl/data.frame)
## $ Major : chr [1:60] "Accounting" "Accounting" "Accounting" "Accounting" ...
## $ Salary: num [1:60] 3676 4324 4131 4005 3963 ...

head(Salary) # Zeigt die ersten Werte der Salary-Variable

## [1] 3676 4324 4131 4005 3963 3317
```

R-Erklärung:

- Die Funktion `library()` lädt ein installiertes Paket in die aktuelle R-Sitzung.
- Mit `read_excel()` aus dem Paket `readxl` können Excel-Dateien eingelesen werden. Der Parameter `sheet` gibt an, welches Tabellenblatt verwendet werden soll.
- Der `$`-Operator wird verwendet, um auf eine Spalte eines Dataframes zuzugreifen.
- Die Funktion `str()` zeigt die Struktur eines Objekts, einschliesslich Datentypen und Dimensionen.
- Die Funktion `head()` zeigt die ersten Zeilen eines Dataframes oder Vektors.

Hinweis zu Datenpfaden: In R-Markdown-Dateien werden relative Pfade vom Speicherort der Datei aus interpretiert, daher der Pfad `"../data/WDDA_03.xlsx"`. In regulären R-Skripten würde der Pfad relativ zum Arbeitsverzeichnis angegeben werden.

3 Lagemasse

Lagemasse beschreiben die zentrale Tendenz einer Verteilung und geben an, wo sich das Zentrum der Daten befindet. Sie sind wichtige Kennzahlen, um einen ersten Eindruck von der Lage der Daten zu bekommen.

3.1 Arithmetischer Mittelwert

Der arithmetische Mittelwert ist die Summe aller Datenwerte geteilt durch die Anzahl der Beobachtungen. Er ist das am häufigsten verwendete Lagemass, hat jedoch den Nachteil, dass er empfindlich gegenüber Ausreißern ist.

Mathematische Formel: $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

```
# Manuelle Berechnung
sum_Salary <- Salary |> sum() # Berechnet die Summe aller Gehälter
n_Salary <- Salary |> length() # Zählt die Anzahl der Beobachtungen
mittelwert_manual <- sum_Salary / n_Salary # Teilt die Summe durch die Anzahl

# Eingebaute Funktion
mittelwert_builtin <- Salary |> mean() # Verwendet die eingebaute mean()-Funktion

# Ausgabe der Ergebnisse
print(paste("Mittelwert (manuell):", round(mittelwert_manual, 2)))
```

```
## [1] "Mittelwert (manuell): 3630.72"
```

```
print(paste("Mittelwert (eingebaut):", round(mittelwert_builtin, 2)))
```

```
## [1] "Mittelwert (eingebaut): 3630.72"
```

```
# Demonstration der Ausreisserempfindlichkeit
salary_with_outlier <- c(Salary, 1000000) # Fügt einen extremen Ausreisser hinzu
print(paste("Mittelwert ohne Ausreisser:", round(mean(Salary), 2)))
```

```
## [1] "Mittelwert ohne Ausreisser: 3630.72"
```

```
print(paste("Mittelwert mit Ausreisser:", round(mean(salary_with_outlier), 2)))
```

```
## [1] "Mittelwert mit Ausreisser: 19964.64"
```

Statistische Erklärung: Der arithmetische Mittelwert ist ein Schwerpunkt der Daten. Er minimiert die Summe der quadrierten Abweichungen und ist daher optimal für symmetrische Verteilungen ohne Ausreisser. Bei schiefen Verteilungen oder bei Vorhandensein von Ausreißern kann der Mittelwert jedoch ein verzerrtes Bild der zentralen Tendenz geben.

R-Erklärung:

- Die Funktion `sum()` berechnet die Summe aller Werte in einem Vektor.
- Die Funktion `length()` gibt die Anzahl der Elemente in einem Vektor zurück.
- Die Funktion `mean()` berechnet direkt den arithmetischen Mittelwert.
- Der Pipe-Operator `|>` leitet das Ergebnis des linken Ausdrucks als erstes Argument an die rechte Funktion weiter.
- Die Funktion `round()` rundet auf eine bestimmte Anzahl von Dezimalstellen.

3.2 Median

Der Median teilt die geordneten Daten in zwei Hälften. Er ist robust gegenüber Ausreißern und daher besonders nützlich für schiefe Verteilungen oder Daten mit extremen Werten.

Mathematische Definition:

- Bei ungerader Anzahl n : Der Wert an Position $(n+1)/2$ in der geordneten Reihe
- Bei gerader Anzahl n : Der Mittelwert der Werte an den Positionen $n/2$ und $n/2+1$

```
# Berechnung des Medians mit der eingebauten Funktion
```

```
median_salary <- Salary |> median()
print(paste("Median:", median_salary))
```

```
## [1] "Median: 3663"
```

```
# Manuelle Berechnung zur Demonstration
```

```
sorted_salary <- sort(Salary) # Sortiert die Gehälter aufsteigend
n <- length(sorted_salary) # Anzahl der Beobachtungen
```

```
# Bestimmt den Median je nach gerader oder ungerader Anzahl
```

```
if (n %% 2 == 1) { # Ungerade Anzahl
  median_manual <- sorted_salary[(n+1)/2]
} else { # Gerade Anzahl
  median_manual <- (sorted_salary[n/2] + sorted_salary[n/2 + 1]) / 2
}
print(paste("Median (manuell):", median_manual))
```

```
## [1] "Median (manuell): 3663"
```

```
# Vergleich von Median und Mittelwert bei Ausreißern
```

```
print(paste("Median mit Ausreisser:", median(salary_with_outlier)))
```

```
## [1] "Median mit Ausreisser: 3676"
```

```
print(paste("Mittelwert mit Ausreisser:", round(mean(salary_with_outlier), 2)))
```

```
## [1] "Mittelwert mit Ausreisser: 19964.64"
```

Statistische Erklärung: Der Median ist ein robustes Lagemass, das die Daten in zwei gleich grosse Hälften teilt. Im Gegensatz zum Mittelwert wird er kaum von Ausreißern beeinflusst, da er nur von der Rangordnung der Daten abhängt, nicht von ihren exakten Werten. Der Median ist besonders geeignet für:

- Schiefe Verteilungen (z.B. Einkommensverteilungen)
- Daten mit Ausreißern
- Ordinalskalierte Daten (bei denen nur die Rangordnung, nicht aber die genauen Abstände bekannt sind)

R-Erklärung:

- Die Funktion `median()` berechnet den Median eines Vektors.
- Die Funktion `sort()` sortiert einen Vektor in aufsteigender Reihenfolge.
- Der Modulo-Operator `%%` gibt den Rest einer Division zurück und wird hier verwendet, um zu prüfen, ob die Anzahl gerade oder ungerade ist.

3.3 Modus

Der Modus ist der am häufigsten vorkommende Wert in einem Datensatz. Er ist besonders nützlich für kategoriale Daten und kann im Gegensatz zu Mittelwert und Median auch für nominalskalierte Daten berechnet werden.

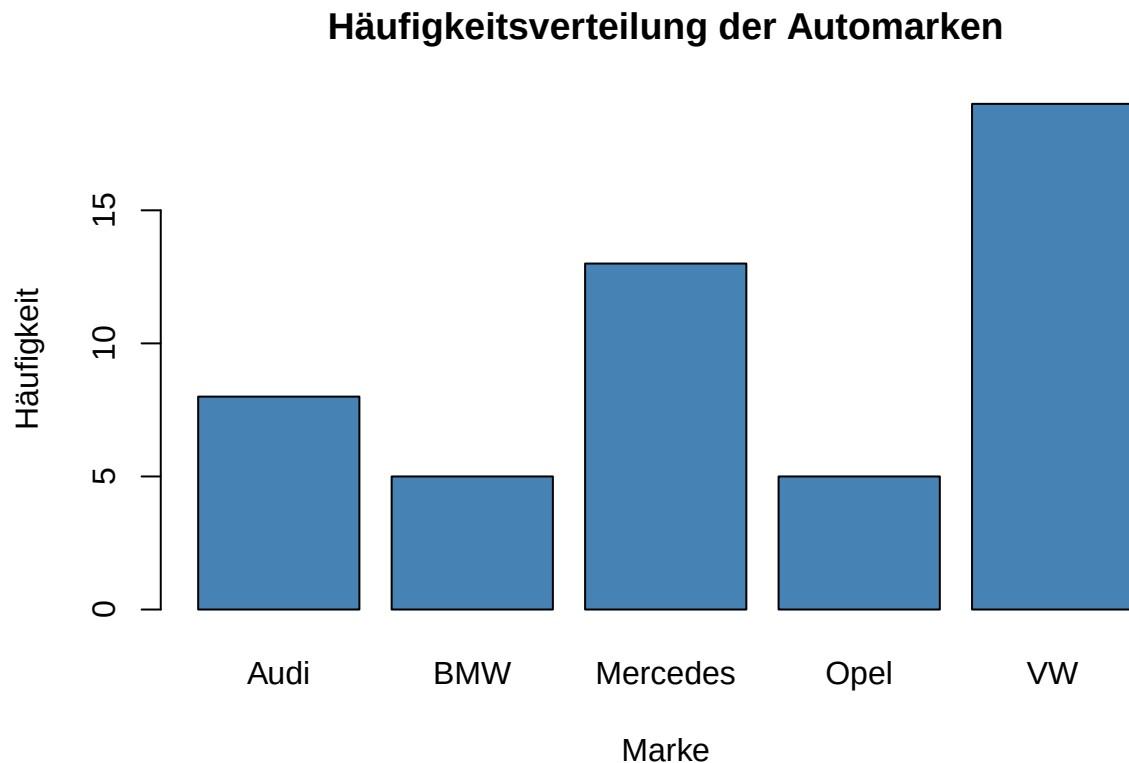
```
# Häufigkeitstabelle erstellen  
brand_table <- Brand |> table()  
print(brand_table)
```

```
## Brand  
##      Audi      BMW Mercedes      Opel      VW  
##        8        5        13        5       19
```

```
# Modus ermitteln  
modus_brand <- brand_table |>  
  which.max() |> # Gibt die Position des grössten Wertes zurück  
  names()        # Extrahiert den Namen (die Kategorie) an dieser Position  
print(paste("Modus:", modus_brand))
```

```
## [1] "Modus: VW"
```

```
# Visualisierung der Häufigkeitsverteilung  
bp <- barplot(brand_table,  
  main = "Häufigkeitsverteilung der Automarken",  
  xlab = "Marke",  
  ylab = "Häufigkeit",  
  col = "steelblue")  
# Markierung des Modus  
text(bp[which(names(brand_table) == modus_brand)],  
  brand_table[modus_brand] + 1,  
  "Modus",  
  col = "red")
```



Statistische Erklärung: Der Modus ist das einzige Lagemass, das für alle Skalenniveaus (nominal, ordinal, intervall, ratio) anwendbar ist. Ein Datensatz kann unimodal (ein Modus), bimodal (zwei Modi) oder multimodal (mehrere Modi) sein. Bei kontinuierlichen Daten wird der Modus oft als der Wert definiert, bei dem die Wahrscheinlichkeitsdichtefunktion ihr Maximum erreicht.

R-Erklärung:

- Die Funktion `table()` erstellt eine Häufigkeitstabelle, die für jede eindeutige Kategorie die Anzahl der Vorkommen zählt.
- Die Funktion `which.max()` gibt den Index (die Position) des grössten Wertes in einem Vektor zurück.
- Die Funktion `names()` extrahiert die Namen eines benannten Vektors oder einer Tabelle.
- R hat keine eingebaute Funktion für den Modus, daher wird er typischerweise mit einer Kombination aus `table()` und `which.max()` berechnet.

Hinweis: Bei mehreren gleich häufigen Werten gibt `which.max()` nur den ersten zurück. Um alle Modi zu finden, müsste man alle Positionen identifizieren, an denen die Häufigkeit dem Maximum entspricht.

3.4 Gewichteter Mittelwert

Beim gewichteten Mittelwert wird jedem Datenwert ein Gewicht zugeordnet, das seine relative Bedeutung widerspiegelt. Dies ist besonders nützlich, wenn nicht alle Beobachtungen gleich wichtig sind oder wenn Daten bereits in aggregierter Form vorliegen.

Mathematische Formel: $\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$

```
# Beispiel: Berechnung des gewichteten Mittelwerts
xvar <- c(3, 3.4, 2.8, 2.9, 3.25) # Werte
weight <- c(1200, 500, 2750, 1000, 800) # Gewichte

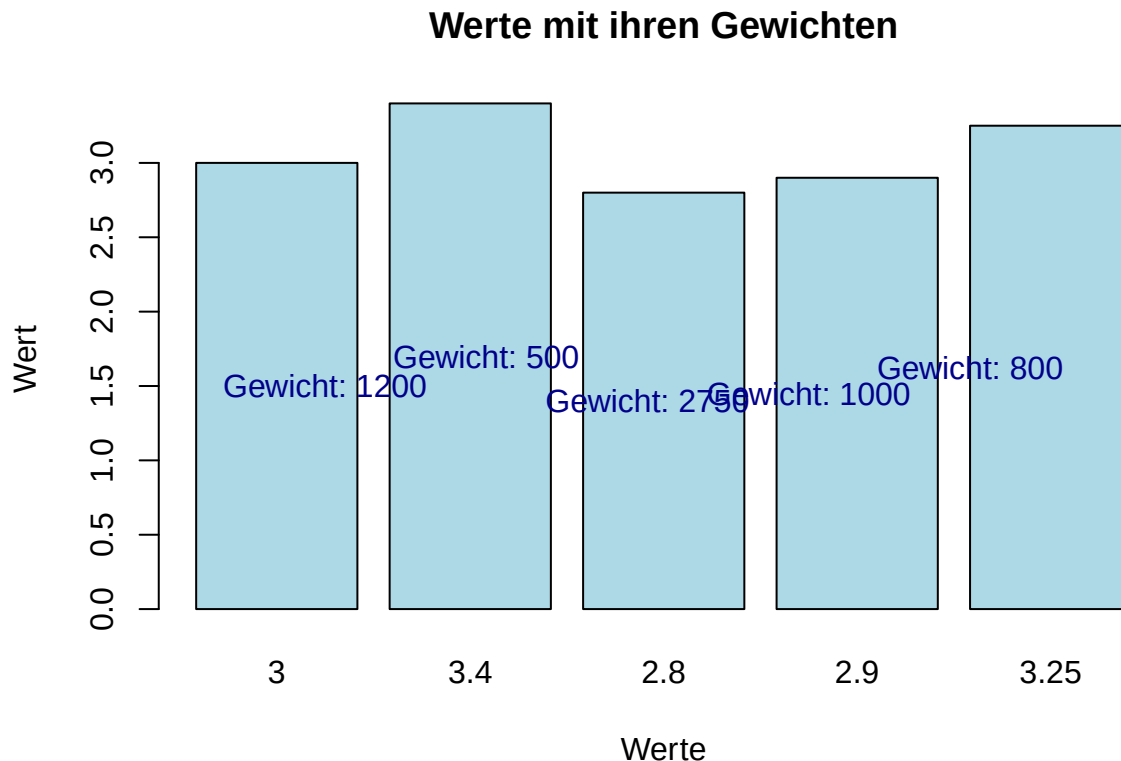
# Berechnung des gewichteten Mittelwerts
gewichteter_mittelwert <- sum(xvar * weight) / sum(weight)
print(paste("Gewichteter Mittelwert:", round(gewichteter_mittelwert, 4)))

## [1] "Gewichteter Mittelwert: 2.96"

# Vergleich mit dem ungewichteten Mittelwert
ungewichteter_mittelwert <- mean(xvar)
print(paste("Ungewichteter Mittelwert:", round(ungewichteter_mittelwert, 4)))

## [1] "Ungewichteter Mittelwert: 3.07"

# Visualisierung der Gewichtung
barplot(xvar,
        names.arg = xvar,
        col = "lightblue",
        main = "Werte mit ihren Gewichten",
        xlab = "Werte",
        ylab = "Wert")
# Füge Gewichte als Text hinzu
text(1:length(xvar), xvar/2,
     labels = paste("Gewicht:", weight),
     col = "darkblue")
```



Statistische Erklärung: Der gewichtete Mittelwert berücksichtigt die unterschiedliche Bedeutung oder Häufigkeit der einzelnen Beobachtungen. Typische Anwendungsfälle sind:

1. **Aggregierte Daten:** Wenn Daten bereits in Gruppen zusammengefasst sind (z.B. Durchschnittsgehälter nach Abteilungen mit unterschiedlicher Mitarbeiterzahl)
2. **Ungleiche Stichprobengrößen:** Wenn verschiedene Teilstichproben unterschiedlich gross sind
3. **Qualitätsunterschiede:** Wenn einige Messungen zuverlässiger sind als andere
4. **Zeitreihen:** Bei der Berechnung gleitender Durchschnitte, bei denen neuere Daten oft stärker gewichtet werden

R-Erklärung:

- Die elementweise Multiplikation von Vektoren in R erfolgt mit dem `*`-Operator.
- Die Funktion `weighted.mean(x, w)` ist eine eingebaute Alternative zur manuellen Berechnung.
- Bei der Visualisierung mit `barplot()` kann der Parameter `names.arg` verwendet werden, um die Balken zu beschriften.

4 Streuungsmaße

Streuungsmaße beschreiben, wie stark die Daten um das Zentrum (Lagemass) verteilt sind. Sie geben Aufschluss über die Variabilität oder Dispersion der Daten und sind wichtig, um die Homogenität oder Heterogenität eines Datensatzes zu beurteilen.

4.1 Spannweite

Die Spannweite (Range) ist die Differenz zwischen dem grössten und dem kleinsten Wert in einem Datensatz. Sie ist das einfachste Streuungsmass, hat jedoch den Nachteil, dass sie nur von zwei extremen Werten abhängt und nichts über die Verteilung dazwischen aussagt.

```
max_salary <- Salary |> max() # Grösster Wert
min_salary <- Salary |> min() # Kleinster Wert
spannweite <- max_salary - min_salary # Differenz
print(paste("Maximum:", max_salary))

## [1] "Maximum: 4568"

print(paste("Minimum:", min_salary))

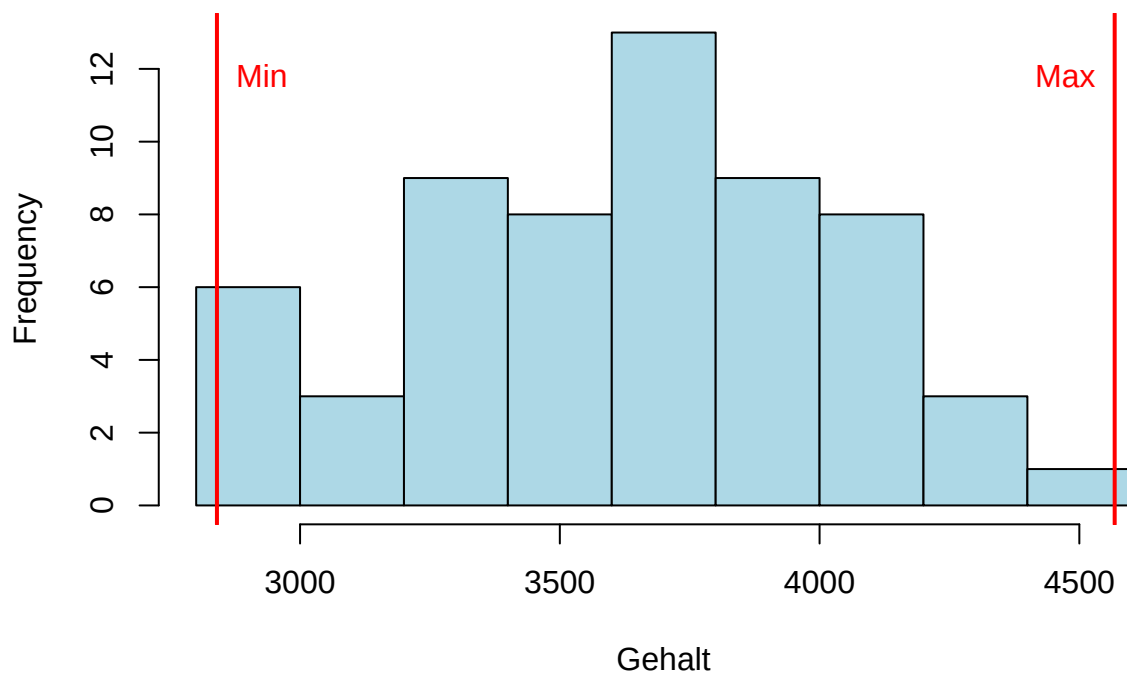
## [1] "Minimum: 2840"

print(paste("Spannweite:", spannweite))

## [1] "Spannweite: 1728"

# Visualisierung der Spannweite
hist(Salary,
     main = "Histogramm der Gehälter mit Spannweite",
     xlab = "Gehalt",
     col = "lightblue")
abline(v = min_salary, col = "red", lwd = 2)
abline(v = max_salary, col = "red", lwd = 2)
text(min_salary, max(hist(Salary, plot = FALSE)$counts) * 0.9,
     "Min", pos = 4, col = "red")
text(max_salary, max(hist(Salary, plot = FALSE)$counts) * 0.9,
     "Max", pos = 2, col = "red")
```

Histogramm der Gehälter mit Spannweite



Statistische Erklärung: Die Spannweite gibt einen ersten Eindruck von der Streuung der Daten, ist aber sehr empfindlich gegenüber Ausreißern. Sie wird oft als ergänzende Information zu robusteren Streuungsmassen wie dem Interquartilsabstand angegeben. Die Spannweite ist besonders nützlich für:

- Einen schnellen Überblick über die Gesamtstreuung
- Die Identifikation möglicher Ausreißer
- Die Festlegung von Achsenbereichen in Grafiken

R-Erklärung:

- Die Funktionen `max()` und `min()` geben den grössten bzw. kleinsten Wert eines Vektors zurück.
- Die Funktion `abline()` mit dem Parameter `v` fügt vertikale Linien zu einem bestehenden Plot hinzu.
- Die Funktion `hist()` mit dem Parameter `plot = FALSE` berechnet die Histogramm-Statistiken, ohne sie zu plotten.

4.2 Quartile und Interquartilsabstand (IQR)

Quartile teilen die geordneten Daten in vier gleiche Teile. Der Interquartilsabstand (IQR) ist die Differenz zwischen dem dritten Quartil (Q3) und dem ersten Quartil (Q1) und umfasst die mittleren 50% der Daten. Er ist ein robustes Streuungsmass, das nicht von Ausreißern beeinflusst wird.

```
# Berechnung der Quartile
Q1 <- Salary |> quantile(0.25) # Erstes Quartil (25. Perzentil)
Q2 <- Salary |> quantile(0.50) # Zweites Quartil = Median (50. Perzentil)
Q3 <- Salary |> quantile(0.75) # Drittes Quartil (75. Perzentil)
```

```
# Berechnung des Interquartilsabstands
IQR_salary <- Q3 - Q1
```

```
print(paste("Q1 (25%):", Q1))
```

```
## [1] "Q1 (25%): 3337.75"
```

```
print(paste("Q2 (Median, 50%):", Q2))
```

```
## [1] "Q2 (Median, 50%): 3663"
```

```
print(paste("Q3 (75%):", Q3))
```

```
## [1] "Q3 (75%): 3969"
```

```
print(paste("IQR:", IQR_salary))
```

```
## [1] "IQR: 631.25"
```

```
# Perzentile
perzentil10 <- Salary |> quantile(0.10) # 10. Perzentil
perzentil95 <- Salary |> quantile(0.95) # 95. Perzentil
print(paste("10. Perzentil:", perzentil10))
```

```
## [1] "10. Perzentil: 3046.2"
```

```
print(paste("95. Perzentil:", perzentil95))
```

```
## [1] "95. Perzentil: 4241.35"
```

```
# Identifikation von Ausreißern mit der 1.5*IQR-Regel
lower_fence <- Q1 - 1.5 * IQR_salary
upper_fence <- Q3 + 1.5 * IQR_salary
print(paste("Untere Grenze für Ausreisser:", lower_fence))
```

```
## [1] "Untere Grenze für Ausreisser: 2390.875"
```

```
print(paste("Obere Grenze für Ausreisser:", upper_fence))
```

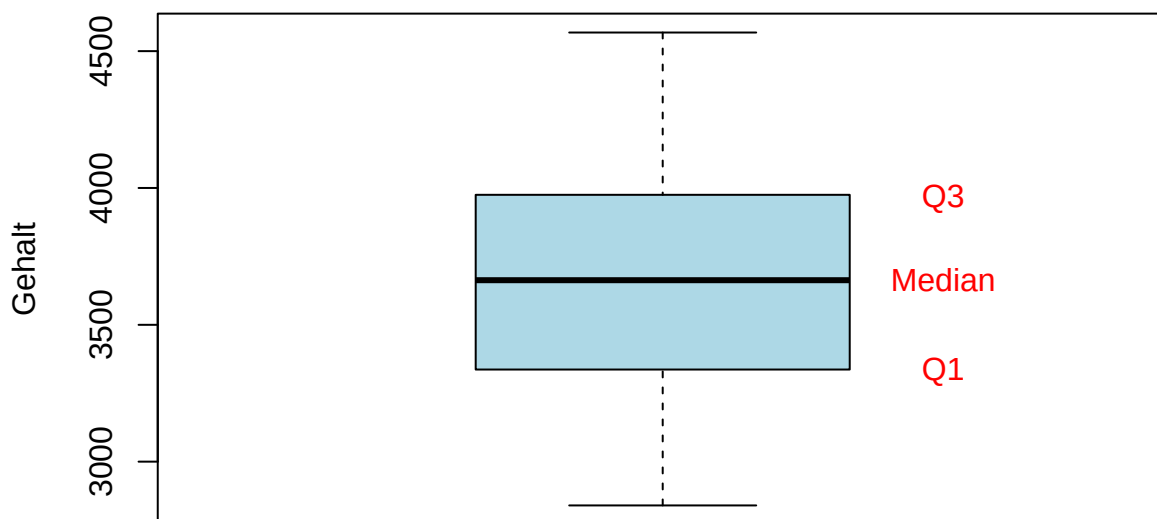
```
## [1] "Obere Grenze für Ausreisser: 4915.875"
```

```
# Anzahl der Ausreisser
outliers <- Salary[Salary < lower_fence | Salary > upper_fence]
print(paste("Anzahl der Ausreisser:", length(outliers)))
```

```
## [1] "Anzahl der Ausreisser: 0"
```

```
# Visualisierung mit Boxplot
boxplot(Salary,
        main = "Boxplot der Gehälter mit Quartilen",
        ylab = "Gehalt",
        col = "lightblue")
text(1.3, Q1, "Q1", col = "red")
text(1.3, Q2, "Median", col = "red")
text(1.3, Q3, "Q3", col = "red")
text(1.3, upper_fence, "Obere Ausreissergrenze", col = "darkred")
text(1.3, lower_fence, "Untere Ausreissergrenze", col = "darkred")
```

Boxplot der Gehälter mit Quartilen



Statistische Erklärung:

- **Quartile** teilen die geordneten Daten in vier gleich grosse Gruppen:
 - Q1 (25. Perzentil): 25% der Daten liegen darunter
 - Q2 (50. Perzentil = Median): 50% der Daten liegen darunter
 - Q3 (75. Perzentil): 75% der Daten liegen darunter
- Der **Interquartilsabstand (IQR)** = $Q3 - Q1$ umfasst die mittleren 50% der Daten
- Die ****1.5*IQR-Regel**** wird häufig verwendet, um Ausreisser zu identifizieren:
 - Werte $< Q1 - 1.5IQR$ oder $> Q3 + 1.5IQR$ gelten als Ausreisser
 - Diese Grenzen werden in Boxplots als “Whiskers” (Antennen) dargestellt

R-Erklärung:

- Die Funktion `quantile()` berechnet Quantile (Perzentile) eines Vektors. Der zweite Parameter gibt den Prozentsatz als Dezimalzahl an.
- Die Funktion `IQR()` ist eine eingebaute Alternative zur manuellen Berechnung des Interquartilsabstands.
- In Boxplots werden Ausreisser typischerweise als einzelne Punkte dargestellt.

4.3 Varianz und Standardabweichung

Die Varianz misst die durchschnittliche quadrierte Abweichung vom Mittelwert und ist ein zentrales Mass für die Streuung der Daten. Die Standardabweichung ist die Quadratwurzel der Varianz und hat den Vorteil, dass sie in der gleichen Einheit wie die Originaldaten ist.

Mathematische Formeln:

- Varianz (Stichprobe): $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$
- Standardabweichung (Stichprobe): $s = \sqrt{s^2}$

```
# Varianz
var_manual <- Salary |>
  (\(x) sum((x - mean(x))^2))() / (length(Salary) - 1) # Anonyme Funktion mit R-Pipe
var_builtin <- Salary |> var() # Eingebaute Funktion

# Standardabweichung
sd_manual <- Salary |> var() |> sqrt() # Quadratwurzel der Varianz
sd_builtin <- Salary |> sd() # Eingebaute Funktion

print(paste("Varianz (manuell):", round(var_manual, 2)))
```

```
## [1] "Varianz (manuell): 168789.77"
```

```
print(paste("Varianz (eingebaut):", round(var_builtin, 2)))
```

```
## [1] "Varianz (eingebaut): 168789.77"
```

```
print(paste("Standardabweichung (manuell):", round(sd_manual, 2)))
```

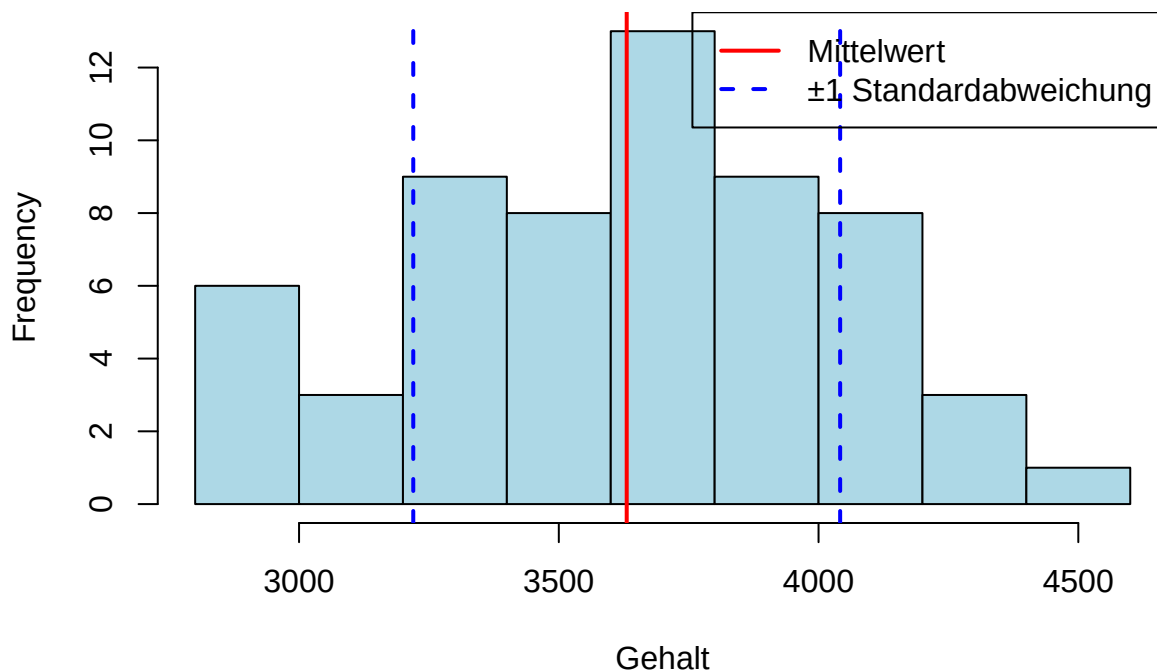
```
## [1] "Standardabweichung (manuell): 410.84"
```

```
print(paste("Standardabweichung (eingebaut):", round(sd_builtin, 2)))
```

```
## [1] "Standardabweichung (eingebaut): 410.84"
```

```
# Visualisierung der Standardabweichung
hist(Salary,
     main = "Histogramm der Gehälter mit Standardabweichung",
     xlab = "Gehalt",
     col = "lightblue")
abline(v = mean(Salary), col = "red", lwd = 2)
abline(v = mean(Salary) + sd(Salary), col = "blue", lwd = 2, lty = 2)
abline(v = mean(Salary) - sd(Salary), col = "blue", lwd = 2, lty = 2)
legend("topright",
     legend = c("Mittelwert", "±1 Standardabweichung"),
     col = c("red", "blue"),
     lwd = 2,
     lty = c(1, 2))
```

Histogramm der Gehälter mit Standardabweichung



Statistische Erklärung:

- Die **Varianz** misst, wie weit die Daten im Durchschnitt vom Mittelwert entfernt sind. Die Abweichungen werden quadriert, um negative und positive Abweichungen gleich zu gewichten.
- Bei der Berechnung der Stichprobenvarianz wird durch $(n-1)$ statt durch n geteilt, um eine unverzerrte Schätzung der Populationsvarianz zu erhalten (Bessel-Korrektur).
- Die **Standardabweichung** ist die Quadratwurzel der Varianz und hat den Vorteil, dass sie in der gleichen Einheit wie die Originaldaten ist, was die Interpretation erleichtert.
- Bei normalverteilten Daten liegen etwa 68% der Werte innerhalb einer Standardabweichung vom Mittelwert (empirische Regel).

R-Erklärung:

- Die anonyme Funktion `\(x) ...` ist eine neue Syntax in R (ab Version 4.1.0) für anonyme Funktionen.
- Die Klammern `()` nach der anonymen Funktion führen diese sofort aus.
- Die Funktionen `var()` und `sd()` berechnen die Stichprobenvarianz bzw. -standardabweichung.

- Der Parameter `lty` in `abline()` steuert den Linientyp (1 = durchgezogen, 2 = gestrichelt).
- Die Funktion `legend()` fügt eine Legende zum Plot hinzu.

Tipp: Die Standardabweichung hat den Vorteil, dass sie in der gleichen Einheit wie die Originaldaten ist, was die Interpretation erleichtert. Sie ist besonders nützlich für:

- Vergleiche zwischen verschiedenen Datensätzen
- Die Beurteilung der Präzision von Messungen
- Die Berechnung von Konfidenzintervallen

5 Empirische Regel

Die empirische Regel (auch 68-95-99,7-Regel genannt) beschreibt die Verteilung der Daten um den Mittelwert bei annähernd normalverteilten Daten. Sie besagt, dass:

- Etwa 68% der Daten innerhalb einer Standardabweichung vom Mittelwert liegen
- Etwa 95% der Daten innerhalb von zwei Standardabweichungen vom Mittelwert liegen
- Etwa 99,7% der Daten innerhalb von drei Standardabweichungen vom Mittelwert liegen

```
# Mittelwert und Standardabweichung
mean_salary <- mean(Salary)
sd_salary <- sd(Salary)

# Intervall [mean - sd, mean + sd] (68%-Regel)
untere_grenze_1sd <- mean_salary - sd_salary
obere_grenze_1sd <- mean_salary + sd_salary
print(paste("Intervall [mean ± 1sd]:", round(untere_grenze_1sd, 2), "bis",
  ↪ round(obere_grenze_1sd, 2)))

## [1] "Intervall [mean ± 1sd]: 3219.88 bis 4041.56"

# Intervall [mean - 2sd, mean + 2sd] (95%-Regel)
untere_grenze_2sd <- mean_salary - 2*sd_salary
obere_grenze_2sd <- mean_salary + 2*sd_salary
print(paste("Intervall [mean ± 2sd]:", round(untere_grenze_2sd, 2), "bis",
  ↪ round(obere_grenze_2sd, 2)))

## [1] "Intervall [mean ± 2sd]: 2809.04 bis 4452.4"

# Intervall [mean - 3sd, mean + 3sd] (99.7%-Regel)
untere_grenze_3sd <- mean_salary - 3*sd_salary
obere_grenze_3sd <- mean_salary + 3*sd_salary
print(paste("Intervall [mean ± 3sd]:", round(untere_grenze_3sd, 2), "bis",
  ↪ round(obere_grenze_3sd, 2)))

## [1] "Intervall [mean ± 3sd]: 2398.2 bis 4863.24"

# Berechnen des Anteils der Daten in diesen Intervallen
anteil_1sd <- mean(Salary >= untere_grenze_1sd & Salary <= obere_grenze_1sd)
anteil_2sd <- mean(Salary >= untere_grenze_2sd & Salary <= obere_grenze_2sd)
anteil_3sd <- mean(Salary >= untere_grenze_3sd & Salary <= obere_grenze_3sd)

print(paste("Anteil der Daten im 1sd-Intervall:", round(anteil_1sd * 100, 1), "%
  ↪ (Erwartung: ~68%)"))
```

```
## [1] "Anteil der Daten im 1sd-Intervall: 70 % (Erwartung: ~68%)"
```

```
print(paste("Anteil der Daten im 2sd-Intervall:", round(anteil_2sd * 100, 1), "%  
↪ (Erwartung: ~95%)"))
```

```
## [1] "Anteil der Daten im 2sd-Intervall: 98.3 % (Erwartung: ~95%)"
```

```
print(paste("Anteil der Daten im 3sd-Intervall:", round(anteil_3sd * 100, 1), "%  
↪ (Erwartung: ~99.7%)"))
```

```
## [1] "Anteil der Daten im 3sd-Intervall: 100 % (Erwartung: ~99.7%)"
```

```
# Visualisierung der empirischen Regel
# Erweitere den Bereich für x, um die vollständige Kurve zu zeigen
x_range <- max(Salary) - min(Salary)
x <- seq(min(Salary) - 0.2*x_range, max(Salary) + 0.2*x_range, length.out = 200)
y <- dnorm(x, mean = mean_salary, sd = sd_salary)

# Berechne das Maximum der Dichte für die y-Achsenkalierung
y_max <- max(y) * 1.1 # 10% Puffer über dem Maximum

plot(x, y, type = "l", lwd = 2, col = "blue",
     main = "Normalverteilung mit empirischer Regel",
     xlab = "Gehalt", ylab = "Dichte",
     xlim = c(min(x), max(x)), # Explizite x-Grenzen setzen
     ylim = c(0, y_max)) # Explizite y-Grenzen setzen

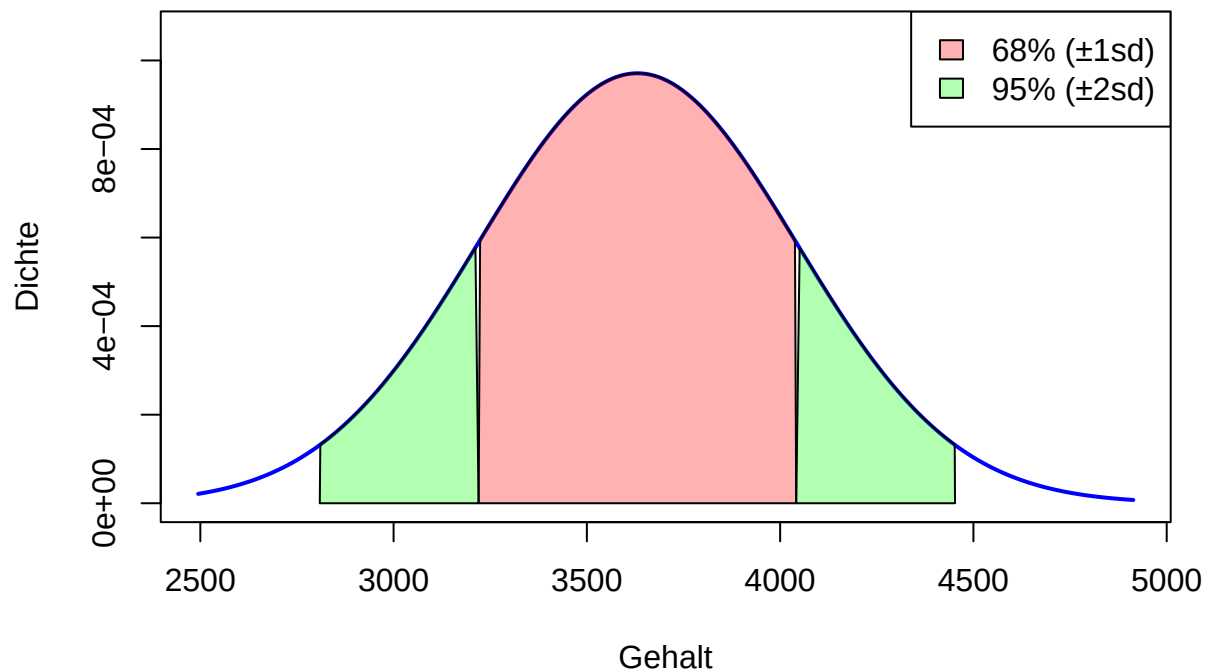
# Füllen der Bereiche unter der Kurve
polygon(c(untere_grenze_1sd, x[x >= untere_grenze_1sd & x <= obere_grenze_1sd],
↪ obere_grenze_1sd),
       c(0, y[x >= untere_grenze_1sd & x <= obere_grenze_1sd], 0),
       col = rgb(1, 0, 0, 0.3)) # 68% (±1sd) in Rot

polygon(c(untere_grenze_2sd, x[x >= untere_grenze_2sd & x < untere_grenze_1sd],
↪ untere_grenze_1sd),
       c(0, y[x >= untere_grenze_2sd & x < untere_grenze_1sd], 0),
       col = rgb(0, 1, 0, 0.3)) # Zusätzliche 13.5% links (±2sd) in Grün

polygon(c(obere_grenze_1sd, x[x > obere_grenze_1sd & x <= obere_grenze_2sd],
↪ obere_grenze_2sd),
       c(0, y[x > obere_grenze_1sd & x <= obere_grenze_2sd], 0),
       col = rgb(0, 1, 0, 0.3)) # Zusätzliche 13.5% rechts (±2sd) in Grün

# Legende
legend("topright",
     legend = c("68% (±1sd)", "95% (±2sd)"),
     fill = c(rgb(1, 0, 0, 0.3), rgb(0, 1, 0, 0.3)))
```

Normalverteilung mit empirischer Regel



Statistische Erklärung: Die empirische Regel ist eine Faustregel für die Verteilung von Daten in einer Normalverteilung. Sie ist nützlich für:

- Die schnelle Abschätzung, welcher Anteil der Daten in bestimmten Bereichen liegt
- Die Identifikation potenzieller Ausreisser (Werte ausserhalb von 3 Standardabweichungen)
- Die Beurteilung, ob Daten annähernd normalverteilt sind (durch Vergleich der tatsächlichen Anteile mit den erwarteten)

Die Abweichungen der tatsächlichen Anteile von den erwarteten Werten (68%, 95%, 99,7%) können Hinweise auf die Form der Verteilung geben:

- Grössere Anteile in den Intervallen deuten auf eine spitzere Verteilung hin (höhere Kurtosis)
- Kleinere Anteile deuten auf schwerere Ränder hin (niedrigere Kurtosis)

R-Erklärung:

- Die Funktion `mean()` mit einem logischen Vektor berechnet den Anteil der TRUE-Werte.
- Die Funktion `dnorm()` berechnet die Dichte der Normalverteilung an gegebenen Punkten.
- Die Funktion `polygon()` füllt einen Bereich unter einer Kurve.
- Die Funktion `rgb()` erstellt Farben mit Transparenz (der vierte Parameter ist Alpha).

6 Standardisierung

Die Standardisierung (z-Transformation) macht verschiedene Variablen vergleichbar, indem sie sie auf eine gemeinsame Skala mit Mittelwert 0 und Standardabweichung 1 bringt. Dies ist besonders nützlich, wenn Variablen mit unterschiedlichen Einheiten oder Grössenordnungen verglichen werden sollen.

Mathematische Formel: $z = \frac{x - \mu}{\sigma}$, wobei μ der Mittelwert und σ die Standardabweichung ist.

```
# Standardisierung der Gehaltsdaten
Salary_z <- (Salary - (Salary |> mean())) / (Salary |> sd())
```

```
head(Salary_z)  # Zeigt die ersten standardisierten Werte

## [1]  0.1102213  1.6874764  1.2177075  0.9110190  0.8087895 -0.7635976

# Überprüfen der Eigenschaften der standardisierten Daten
print(paste("Mittelwert der z-Werte:", round(mean(Salary_z), 10)))  # Sollte nahe 0 sein

## [1] "Mittelwert der z-Werte: 0"

print(paste("Standardabweichung der z-Werte:", round(sd(Salary_z), 10)))  # Sollte nahe 1
↪ sein

## [1] "Standardabweichung der z-Werte: 1"

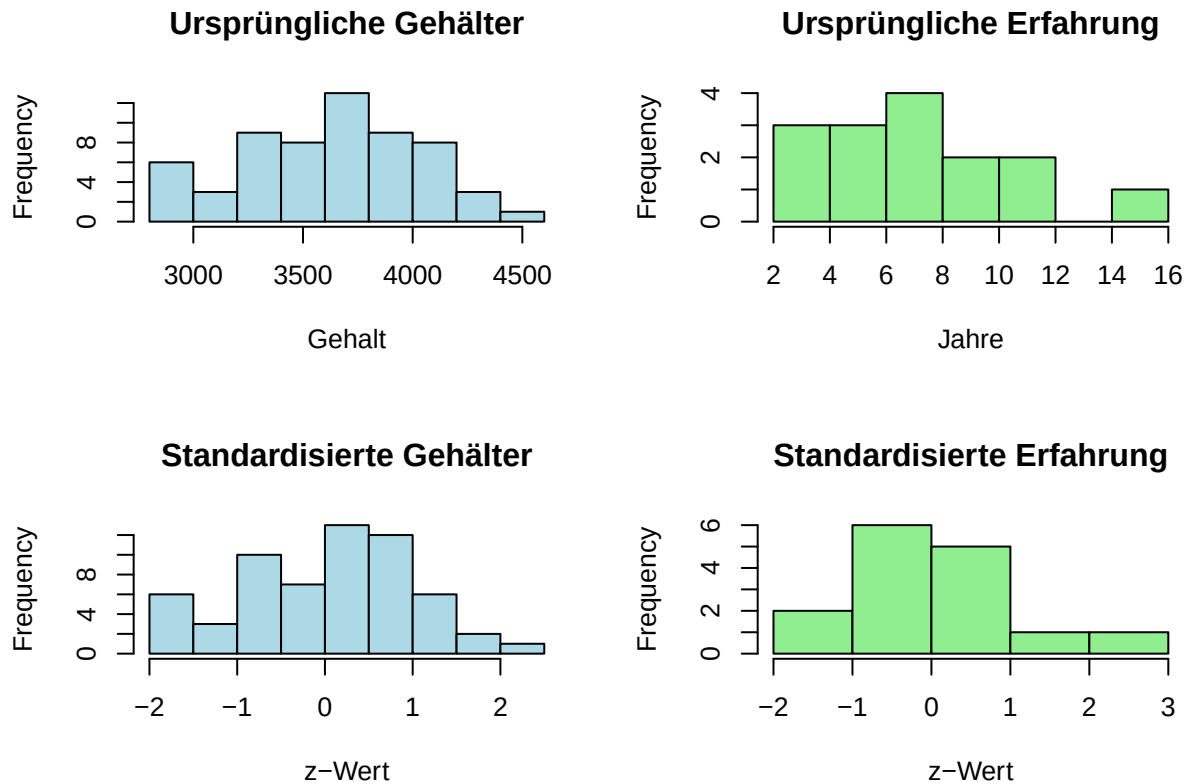
# Beispiel: Standardisierung einer zweiten Variable zum Vergleich
# Angenommen, wir haben auch Daten über Berufserfahrung in Jahren
Experience <- c(5, 3, 8, 12, 7, 10, 15, 2, 6, 9, 11, 4, 8, 7, 5)

# Standardisierung beider Variablen
Experience_z <- (Experience - mean(Experience)) / sd(Experience)

# Vergleich der ursprünglichen und standardisierten Werte
par(mfrow = c(2, 2))  # 2x2 Plotanordnung

# Ursprüngliche Werte
hist(Salary, main = "Ursprüngliche Gehälter", xlab = "Gehalt", col = "lightblue")
hist(Experience, main = "Ursprüngliche Erfahrung", xlab = "Jahre", col = "lightgreen")

# Standardisierte Werte
hist(Salary_z, main = "Standardisierte Gehälter", xlab = "z-Wert", col = "lightblue")
hist(Experience_z, main = "Standardisierte Erfahrung", xlab = "z-Wert", col =
↪ "lightgreen")
```

```
# Zurücksetzen auf ein einzelnes Plot
par(mfrow = c(1, 1))

# Interpretation der z-Werte
# Beispiel: Identifikation von Werten, die mehr als 2 Standardabweichungen vom Mittelwert
#           entfernt sind
extreme_values <- Salary[abs(Salary_z) > 2]
print(paste("Anzahl extremer Werte (|z| > 2):", length(extreme_values)))

## [1] "Anzahl extremer Werte (|z| > 2): 1"

if (length(extreme_values) > 0) {
  print("Extreme Werte:")
  print(extreme_values)
}

## [1] "Extreme Werte:"
## [1] 4568
```

Statistische Erklärung: Die Standardisierung (z-Transformation) hat mehrere wichtige Anwendungen:

1. **Vergleichbarkeit:** Variablen mit unterschiedlichen Einheiten oder Grössenordnungen können direkt verglichen werden.
2. **Interpretation:** z-Werte geben an, wie viele Standardabweichungen ein Wert vom Mittelwert entfernt ist.
 - $z = 0$: Der Wert entspricht genau dem Mittelwert
 - $z = 1$: Der Wert liegt eine Standardabweichung über dem Mittelwert
 - $z = -1$: Der Wert liegt eine Standardabweichung unter dem Mittelwert

3. **Ausreissererkennung:** Werte mit $|z| > 2$ oder $|z| > 3$ werden oft als potenzielle Ausreisser betrachtet.
4. **Voraussetzung für viele statistische Verfahren:** Viele multivariate Verfahren setzen standardisierte Variablen voraus.

R-Erklärung:

- Die Standardisierung kann manuell mit der Formel $(x - \text{mean}(x)) / \text{sd}(x)$ durchgeführt werden.
- Die Funktion `scale()` ist eine eingebaute Alternative: `scale(x)` standardisiert einen Vektor oder die Spalten einer Matrix.
- Die Funktion `par(mfrow = c(rows, cols))` teilt das Grafikfenster in eine Matrix mit `rows` Zeilen und `cols` Spalten.
- Die Funktion `abs()` berechnet den Absolutbetrag eines Wertes.

7 Schiefe

Die Schiefe (Skewness) misst die Asymmetrie der Verteilung. Sie gibt an, ob die Verteilung symmetrisch ist oder ob sie einen längeren Schwanz auf einer Seite hat.

Mathematische Formel für die Stichprobenschiefe: $g_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3$

```
# Manuelle Berechnung der Schiefe mit standardisierten Werten
n <- Salary_z |> length()
schiefe <- n / ((n - 1) * (n - 2)) * (Salary_z^3 |> sum())
print(paste("Schiefe (manuell):", round(schiefe, 4)))
```

```
## [1] "Schiefe (manuell): -0.1395"
```

```
# Alternative mit dem Paket e1071
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
library(e1071)
schiefe_e1071 <- skewness(Salary)
print(paste("Schiefe (mit e1071):", round(schiefe_e1071, 4)))
```

```
## [1] "Schiefe (mit e1071): -0.1326"
```

```
# Visualisierung verschiedener Schiefe-Typen
par(mfrow = c(1, 3))

# Rechtsschiefe Verteilung (positive Schiefe)
x_right <- rbeta(1000, 2, 5) * 100 # Beta-Verteilung mit positiver Schiefe
hist(x_right,
     main = "Rechtsschiefe Verteilung\n(positive Schiefe)",
     xlab = "Wert",
     col = "lightblue")
text(60, 200, paste("Schiefe:", round(skewness(x_right), 2)), col = "red")

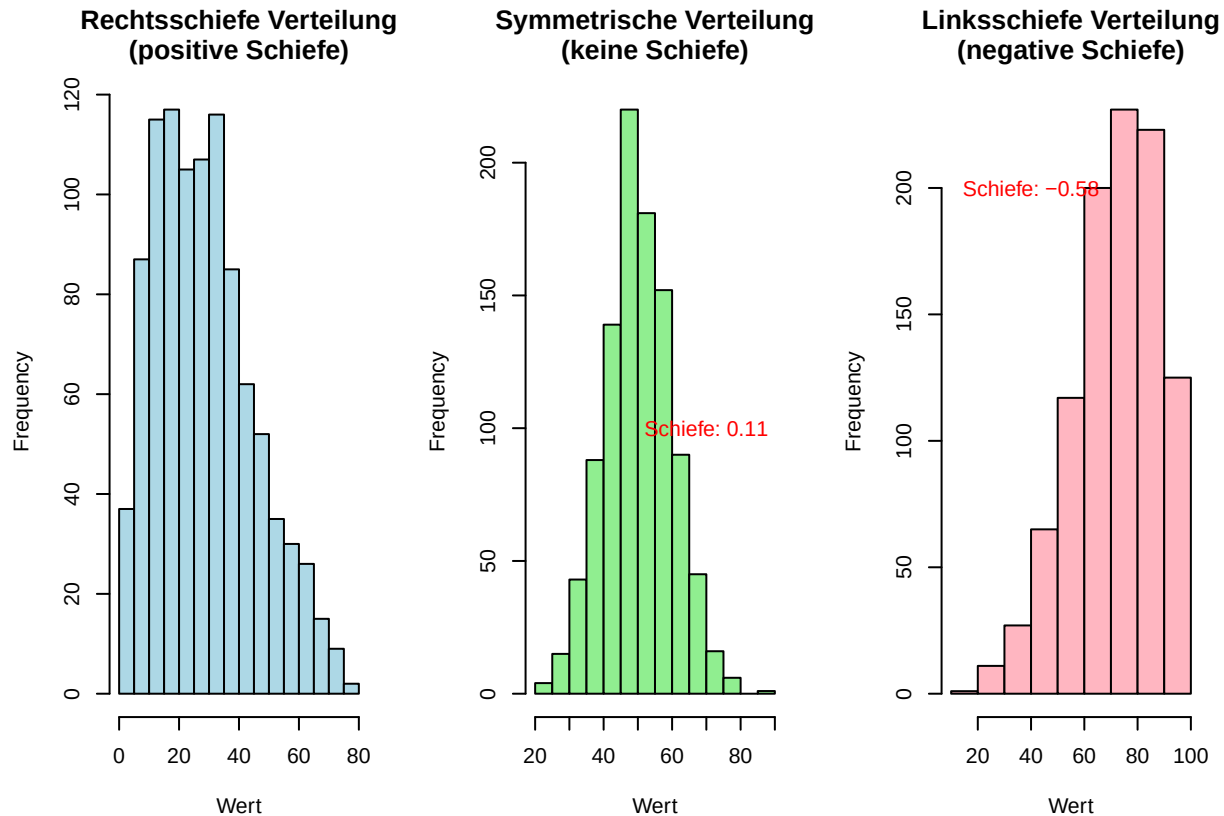
# Symmetrische Verteilung (keine Schiefe)
x_symm <- rnorm(1000, 50, 10) # Normalverteilung (symmetrisch)
hist(x_symm,
     main = "Symmetrische Verteilung\n(keine Schiefe)",
     xlab = "Wert",
     col = "lightgreen")
```

```

text(70, 100, paste("Schiefe:", round(skewness(x_symm), 2)), col = "red")

# Linksschiefe Verteilung (negative Schiefe)
x_left <- 100 - rbeta(1000, 2, 5) * 100 # Umgekehrte Beta-Verteilung
hist(x_left,
     main = "Linksschiefe Verteilung\n(negative Schiefe)",
     xlab = "Wert",
     col = "lightpink")
text(40, 200, paste("Schiefe:", round(skewness(x_left), 2)), col = "red")

```



```

# Zurücksetzen auf ein einzelnes Plot
par(mfrow = c(1, 1))

# Interpretation der Schiefe der Gehaltsdaten
if (schiefe > 0.5) {
  print("Die Gehaltsdaten sind deutlich rechtsschief (positive Schiefe).")
  print("Dies bedeutet, dass es einige wenige hohe Gehälter gibt, die den Mittelwert nach
  ↪ oben ziehen.")
  print("In diesem Fall ist der Median oft aussagekräftiger als der Mittelwert.")
} else if (schiefe < -0.5) {
  print("Die Gehaltsdaten sind deutlich linksschief (negative Schiefe).")
  print("Dies bedeutet, dass es einige wenige niedrige Gehälter gibt, die den Mittelwert
  ↪ nach unten ziehen.")
} else {
  print("Die Gehaltsdaten sind annähernd symmetrisch.")
  print("Mittelwert und Median liegen nahe beieinander.")
}

```

```
}
```

```
## [1] "Die Gehaltsdaten sind annähernd symmetrisch."
## [1] "Mittelwert und Median liegen nahe beieinander."
```

Statistische Erklärung: Die Schiefe ist ein Mass für die Asymmetrie einer Verteilung:

- **Positive Schiefe (> 0):** Rechtsschiefe Verteilung mit einem längeren rechten Schwanz. Der Mittelwert ist grösser als der Median, da er von den hohen Werten im rechten Schwanz nach oben gezogen wird.
- **Negative Schiefe (< 0):** Linksschiefe Verteilung mit einem längeren linken Schwanz. Der Mittelwert ist kleiner als der Median, da er von den niedrigen Werten im linken Schwanz nach unten gezogen wird.
- **Keine Schiefe (= 0):** Symmetrische Verteilung, bei der Mittelwert und Median annähernd gleich sind.

Richtwerte für die Interpretation der Schiefe:

- $|Schiefe| < 0.5$: annähernd symmetrisch
- $0.5 \leq |Schiefe| < 1$: mässig schief
- $|Schiefe| \geq 1$: stark schief

Die Schiefe ist wichtig für die Wahl geeigneter statistischer Methoden, da viele Verfahren eine symmetrische Verteilung voraussetzen.

R-Erklärung:

- Die Funktion `skewness()` aus dem Paket `e1071` berechnet die Schiefe eines numerischen Vektors.
- Die Funktionen `rbeta()` und `rnorm()` erzeugen Zufallszahlen aus der Beta- bzw. Normalverteilung.
- Die Beta-Verteilung mit Parametern `shape1 < shape2` ist rechtsschief, mit `shape1 > shape2` links-schief.
- Die Normalverteilung ist immer symmetrisch (Schiefe = 0).

8 Zusammenhangsmasse

Zusammenhangsmasse quantifizieren die Stärke und Richtung der Beziehung zwischen zwei oder mehr Variablen. Sie sind grundlegend für die Analyse von Abhängigkeiten und Korrelationen in Daten.

8.1 Kovarianz

Die Kovarianz misst den linearen Zusammenhang zwischen zwei Variablen. Sie gibt an, ob die Variablen tendenziell gemeinsam steigen und fallen (positive Kovarianz) oder gegenläufig sind (negative Kovarianz).

Mathematische Formel für die Stichprobenkovarianz: $\text{cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$

```
# Überprüfen der ersten Werte der Variablen
print("Erste Werte von 'sales':")
```

```
## [1] "Erste Werte von 'sales':"
```

```
head(sales)
```

```
## [1] 50 57 41 54 54 38
```

```
print("Erste Werte von 'adverts':")
```

```
## [1] "Erste Werte von 'adverts':"
```

```
head(adverts)
```

```
## [1] 2 5 1 3 4 1
```

```
# Manuelle Berechnung der Kovarianz
n_sales <- sales |> length()
cov_manual <- 1 / (n_sales - 1) * sum((sales - (sales |> mean())) *
                                     (adverts - (adverts |> mean())))
```

```
# Berechnung mit der eingebauten Funktion
cov_builtin <- cov(sales, adverts)
```

```
print(paste("Kovarianz (manuell):", round(cov_manual, 2)))
```

```
## [1] "Kovarianz (manuell): 8.83"
```

```
print(paste("Kovarianz (eingebaut):", round(cov_builtin, 2)))
```

```
## [1] "Kovarianz (eingebaut): 8.83"
```

```
# Visualisierung der Kovarianz
# Grösseres Plotfenster für bessere Lesbarkeit
par(mar = c(5, 4, 4, 2) + 0.5) # Erhöht die Ränder des Plots
```

```
plot(adverts, sales,
     main = "Streudiagramm: Verkäufe vs. Werbung",
     xlab = "Werbeausgaben",
     ylab = "Verkäufe",
     pch = 19,
     col = "blue")
```

```
# Berechnung der Mittelwerte
mean_sales <- mean(sales)
mean_adverts <- mean(adverts)
```

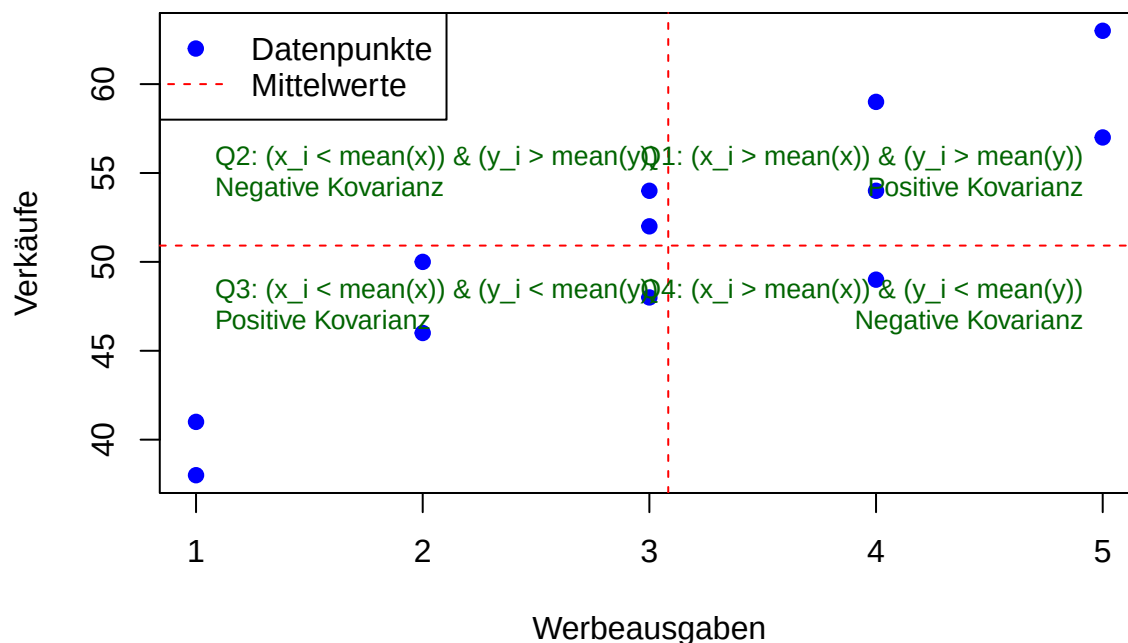
```
# Zeichnen der Mittelwertlinien
abline(h = mean_sales, col = "red", lty = 2)
abline(v = mean_adverts, col = "red", lty = 2)
```

```
# Einteilung des Plots in Quadranten mit besserer Positionierung und kleinerer Schrift
text(min(adverts), mean_sales + 0.15*(max(sales)-min(sales)),
     "Q2: (x_i < mean(x)) & (y_i > mean(y))\nNegative Kovarianz",
     pos = 4, col = "darkgreen", cex = 0.8)
text(max(adverts), mean_sales + 0.15*(max(sales)-min(sales)),
     "Q1: (x_i > mean(x)) & (y_i > mean(y))\nPositive Kovarianz",
     pos = 2, col = "darkgreen", cex = 0.8)
text(min(adverts), mean_sales - 0.15*(max(sales)-min(sales)),
     "Q3: (x_i < mean(x)) & (y_i < mean(y))\nPositive Kovarianz",
     pos = 4, col = "darkgreen", cex = 0.8)
text(max(adverts), mean_sales - 0.15*(max(sales)-min(sales)),
     "Q4: (x_i > mean(x)) & (y_i < mean(y))\nNegative Kovarianz",
     pos = 2, col = "darkgreen", cex = 0.8)
```

```
# Zurücksetzen der Plotparameter nach diesem Plot
par(mar = c(5, 4, 4, 2)) # Standardwerte wiederherstellen

# Hinzufügen einer Legende
legend("topleft",
      legend = c("Datenpunkte", "Mittelwerte"),
      col = c("blue", "red"),
      pch = c(19, NA),
      lty = c(NA, 2))
```

Streudiagramm: Verkäufe vs. Werbung



Statistische Erklärung: Die Kovarianz misst, wie zwei Variablen gemeinsam variieren:

- **Positive Kovarianz:** Die Variablen tendieren dazu, gemeinsam zu steigen und zu fallen. Wenn eine Variable über ihrem Mittelwert liegt, liegt die andere tendenziell auch über ihrem Mittelwert.
- **Negative Kovarianz:** Die Variablen tendieren dazu, in entgegengesetzte Richtungen zu variieren. Wenn eine Variable über ihrem Mittelwert liegt, liegt die andere tendenziell unter ihrem Mittelwert.
- **Kovarianz nahe 0:** Es gibt keinen linearen Zusammenhang zwischen den Variablen.

Die Kovarianz hat jedoch einen Nachteil: Ihr Wert hängt von den Einheiten der Variablen ab, was den Vergleich zwischen verschiedenen Variablenpaaren erschwert. Dieses Problem wird durch den Korrelationskoeffizienten gelöst.

R-Erklärung:

- Die Funktion `cov()` berechnet die Kovarianz zwischen zwei Vektoren.
- Die Funktion `abline()` mit den Parametern `h` und `v` fügt horizontale bzw. vertikale Linien zu einem Plot hinzu.
- Der Parameter `lty` steuert den Linientyp (1 = durchgezogen, 2 = gestrichelt, etc.).
- Die Funktion `text()` fügt Text an bestimmten Koordinaten hinzu.

8.2 Korrelationskoeffizient

Der Korrelationskoeffizient (Pearson's r) normiert die Kovarianz auf den Bereich [-1, 1], indem er sie durch das Produkt der Standardabweichungen teilt. Dies macht ihn unabhängig von den Einheiten der Variablen und ermöglicht direkte Vergleiche zwischen verschiedenen Variablenpaaren.

$$\text{Mathematische Formel: } r = \frac{\text{cov}(X,Y)}{s_X \cdot s_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

```
# Manuelle Berechnung des Korrelationskoeffizienten
corr_manual <- cov(sales, adverts) / ((sales |> sd()) * (adverts |> sd()))

# Berechnung mit der eingebauten Funktion
corr_builtin <- cor(sales, adverts)

print(paste("Korrelation (manuell):", round(corr_manual, 4)))

## [1] "Korrelation (manuell): 0.8884"

print(paste("Korrelation (eingebaut):", round(corr_builtin, 4)))

## [1] "Korrelation (eingebaut): 0.8884"

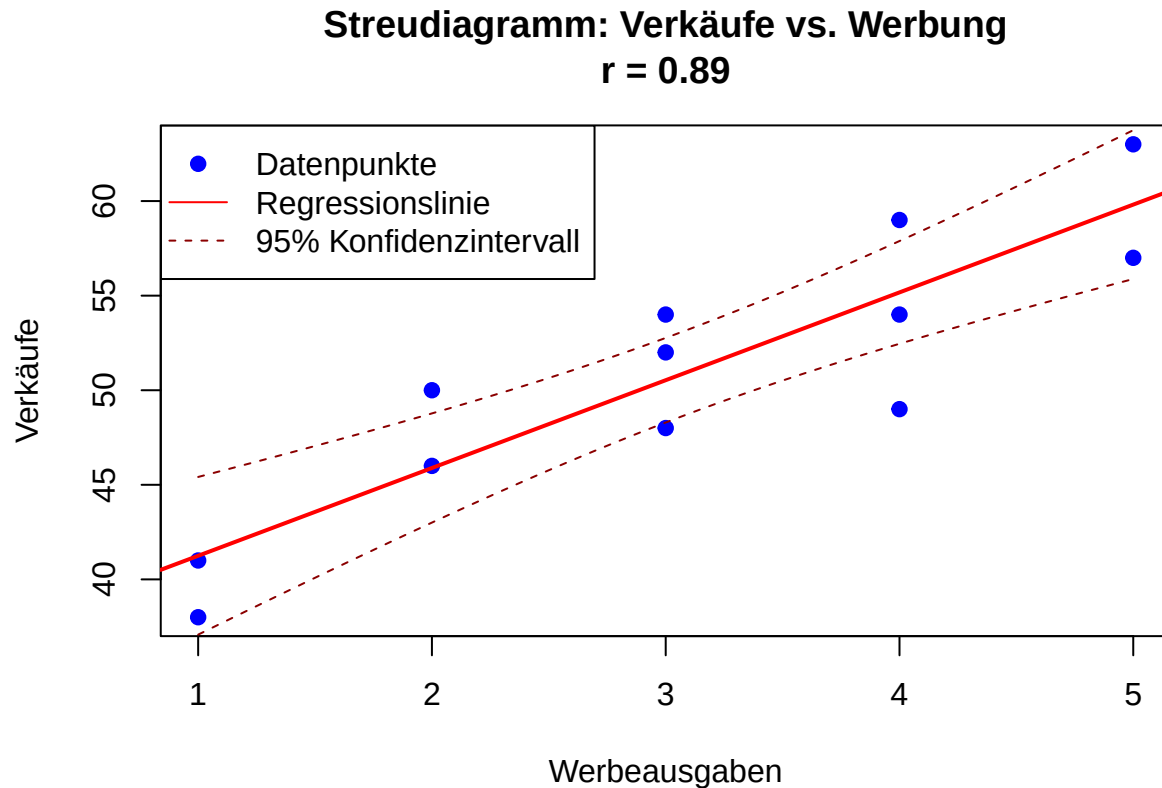
# Visualisierung des Zusammenhangs
plot(adverts, sales,
     main = paste("Streudiagramm: Verkäufe vs. Werbung\nr =", round(corr_builtin, 2)),
     xlab = "Werbeausgaben",
     ylab = "Verkäufe",
     pch = 19,
     col = "blue")

# Hinzufügen einer Regressionslinie
fit <- lm(sales ~ adverts)
abline(fit, col = "red", lwd = 2)

# Konfidenzintervall für die Regressionslinie
newx <- seq(min(adverts), max(adverts), length.out = 100)
conf_interval <- predict(fit, newdata = data.frame(adverts = newx), interval =
  ↪ "confidence", level = 0.95)

lines(newx, conf_interval[, "lwr"], col = "darkred", lty = 2)
lines(newx, conf_interval[, "upr"], col = "darkred", lty = 2)

# Hinzufügen einer Legende
legend("topleft",
     legend = c("Datenpunkte", "Regressionslinie", "95% Konfidenzintervall"),
     col = c("blue", "red", "darkred"),
     pch = c(19, NA, NA),
     lty = c(NA, 1, 2))
```



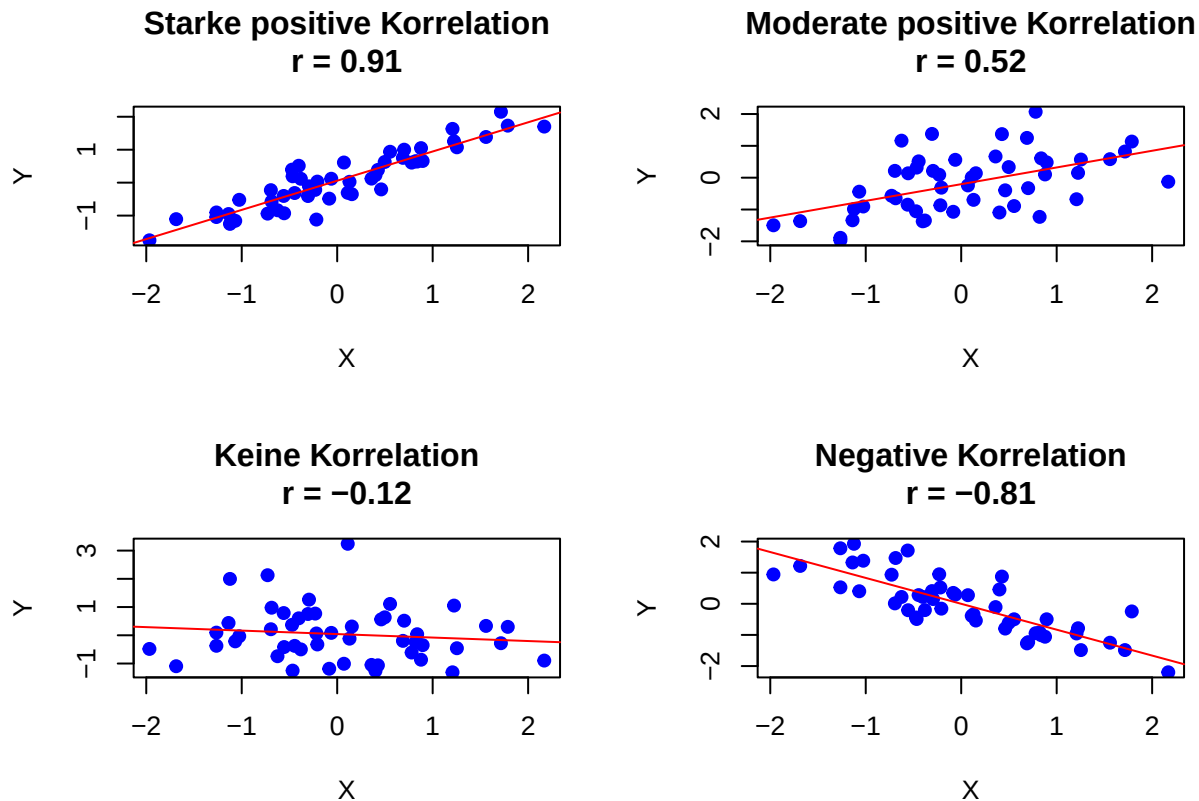
```
# Demonstration verschiedener Korrelationsstärken
par(mfrow = c(2, 2))

# Starke positive Korrelation (r 0.9)
set.seed(123)
x1 <- rnorm(50)
y1 <- 0.9*x1 + rnorm(50, 0, 0.4)
plot(x1, y1, main = paste("Starke positive Korrelation\nr =", round(cor(x1, y1), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y1 ~ x1), col = "red")

# Moderate positive Korrelation (r 0.5)
y2 <- 0.5*x1 + rnorm(50, 0, 0.8)
plot(x1, y2, main = paste("Moderate positive Korrelation\nr =", round(cor(x1, y2), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y2 ~ x1), col = "red")

# Keine Korrelation (r 0)
y3 <- rnorm(50)
plot(x1, y3, main = paste("Keine Korrelation\nr =", round(cor(x1, y3), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y3 ~ x1), col = "red")

# Negative Korrelation (r -0.7)
y4 <- -0.7*x1 + rnorm(50, 0, 0.6)
plot(x1, y4, main = paste("Negative Korrelation\nr =", round(cor(x1, y4), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y4 ~ x1), col = "red")
```

```
# Zurücksetzen auf ein einzelnes Plot
par(mfrow = c(1, 1))
```

Statistische Erklärung: Der Korrelationskoeffizient (Pearson's r) misst die Stärke und Richtung des linearen Zusammenhangs zwischen zwei Variablen:

- $r = 1$: Perfekter positiver linearer Zusammenhang. Alle Datenpunkte liegen exakt auf einer steigenden Geraden.
- $r = -1$: Perfekter negativer linearer Zusammenhang. Alle Datenpunkte liegen exakt auf einer fallenden Geraden.
- $r = 0$: Kein linearer Zusammenhang. Die Variablen sind linear unabhängig.

Richtwerte für die Interpretation der Korrelationsstärke:

- $|r| < 0.3$: schwache Korrelation
- $0.3 \leq |r| < 0.7$: moderate Korrelation
- $|r| \geq 0.7$: starke Korrelation

Wichtige Hinweise zur Interpretation:

1. Korrelation impliziert nicht Kausalität. Ein starker Zusammenhang bedeutet nicht, dass eine Variable die andere verursacht.
2. Der Korrelationskoeffizient misst nur lineare Zusammenhänge. Nichtlineare Beziehungen können trotz $r \neq 0$ stark sein.
3. Der Korrelationskoeffizient ist empfindlich gegenüber Ausreißern.
4. Der Determinationskoeffizient r^2 gibt den Anteil der Varianz in einer Variable an, der durch die andere Variable erklärt wird.

R-Erklärung:

- Die Funktion `cor()` berechnet den Pearson-Korrelationskoeffizienten zwischen zwei Vektoren.

- Mit `method = "spearman"` oder `method = "kendall"` können auch Rangkorrelationen berechnet werden.
- Die Funktion `predict()` mit `interval = "confidence"` berechnet Konfidenzintervalle für die Vorhersagen eines linearen Modells.
- Die Funktion `set.seed()` setzt den Startwert für den Zufallszahlengenerator, um reproduzierbare Ergebnisse zu erhalten.
- Die Funktion `rnorm()` erzeugt normalverteilte Zufallszahlen.

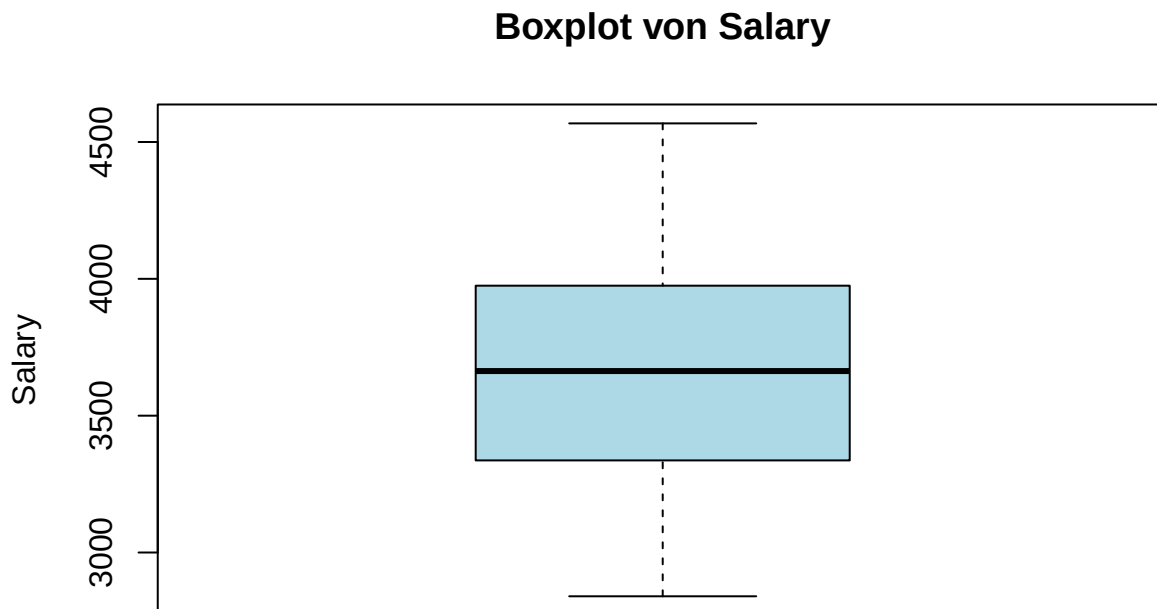
9 Visualisierung

Visualisierungen sind ein mächtiges Werkzeug in der deskriptiven Statistik, um Muster, Trends und Beziehungen in Daten zu erkennen und zu kommunizieren. Sie ergänzen numerische Masse und helfen, komplexe Zusammenhänge intuitiv zu erfassen.

9.1 Boxplots

Boxplots (auch Box-Whisker-Plots genannt) zeigen die Verteilung der Daten über ihre Quartile. Sie sind besonders nützlich zum Erkennen von Ausreißern und zum Vergleich von Gruppen.

```
# Einfacher Boxplot
boxplot(Salary,
        main = "Boxplot von Salary",
        ylab = "Salary",
        col = "lightblue",
        horizontal = FALSE, # Vertikale Ausrichtung
        notch = FALSE) # Ohne Einkerbung
```

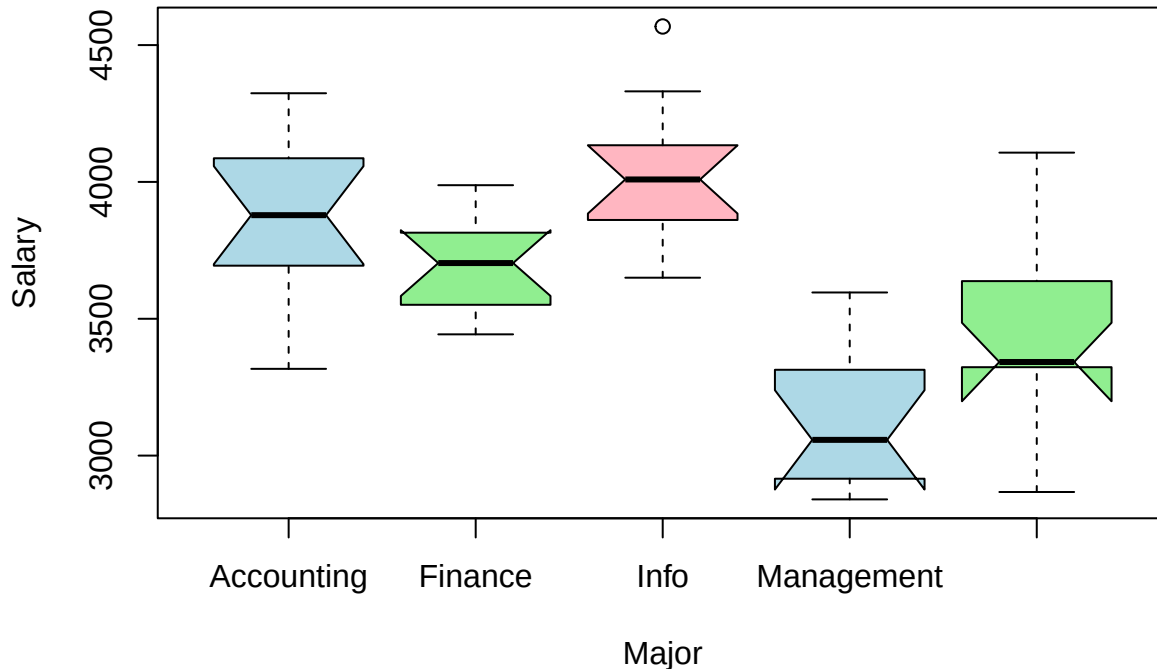


```
# Gruppiertes Boxplot
boxplot(Salary ~ Major,
        data = Graduates,
        main = "Boxplot von Salary nach Major",
        xlab = "Major",
```

```
ylab = "Salary",
col = c("lightblue", "lightgreen", "lightpink"),
notch = TRUE) # Mit Einkerbung für Medianvergleich
```

```
## Warning in (function (z, notch = FALSE, width = NULL, varwidth = FALSE, : some
## notches went outside hinges ('box'): maybe set notch=FALSE
```

Boxplot von Salary nach Major



```
# Erklärung der Boxplot-Elemente mit detaillierten Beschriftungen
bp <- boxplot(Salary,
  main = "Anatomie eines Boxplots",
  ylab = "Salary",
  col = "lightblue",
  outline = TRUE, # Zeigt Ausreisser als Punkte
  plot = TRUE)   # Erzeugt den Plot

# Extrahieren der Boxplot-Statistiken
stats <- bp$stats
outliers <- bp$out

# Beschriftung der Boxplot-Elemente
text(1.3, stats[5, 1], "Maximum (ohne Ausreisser)", pos = 4, col = "darkblue")
text(1.3, stats[4, 1], "Oberes Quartil (Q3, 75%)", pos = 4, col = "darkblue")
text(1.3, stats[3, 1], "Median (Q2, 50%)", pos = 4, col = "darkblue")
text(1.3, stats[2, 1], "Unteres Quartil (Q1, 25%)", pos = 4, col = "darkblue")
text(1.3, stats[1, 1], "Minimum (ohne Ausreisser)", pos = 4, col = "darkblue")

# Beschriftung der Ausreisser, falls vorhanden
if (length(outliers) > 0) {
```

```

text(rep(1.1, length(outliers)), outliers, "Ausreisser", pos = 4, col = "red")
}

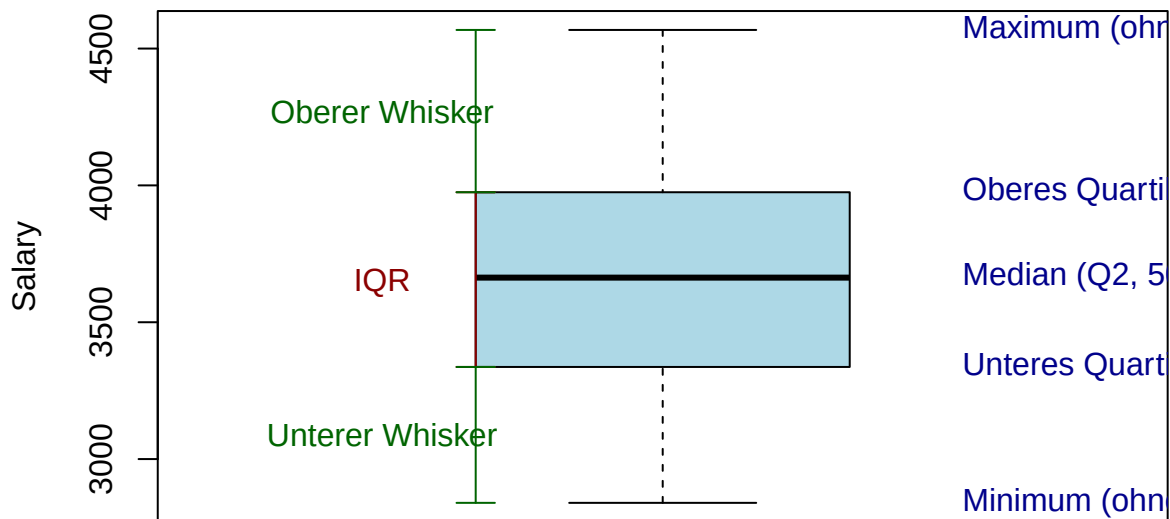
# Hinzufügen von Linien für IQR und Whiskers
arrows(0.8, stats[2, 1], 0.8, stats[4, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkred")
text(0.7, (stats[2, 1] + stats[4, 1])/2, "IQR", col = "darkred")

arrows(0.8, stats[1, 1], 0.8, stats[2, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkgreen")
text(0.7, (stats[1, 1] + stats[2, 1])/2, "Unterer Whisker", col = "darkgreen")

arrows(0.8, stats[4, 1], 0.8, stats[5, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkgreen")
text(0.7, (stats[4, 1] + stats[5, 1])/2, "Oberer Whisker", col = "darkgreen")

```

Anatomie eines Boxplots



```

# Vergleich von Boxplots mit anderen Visualisierungen
par(mfrow = c(1, 3))

```

```

# Boxplot
boxplot(Salary,
  main = "Boxplot",
  ylab = "Salary",
  col = "lightblue")

```

```

# Histogramm
hist(Salary,
  main = "Histogramm",
  xlab = "Salary",
  col = "lightgreen")

```

```

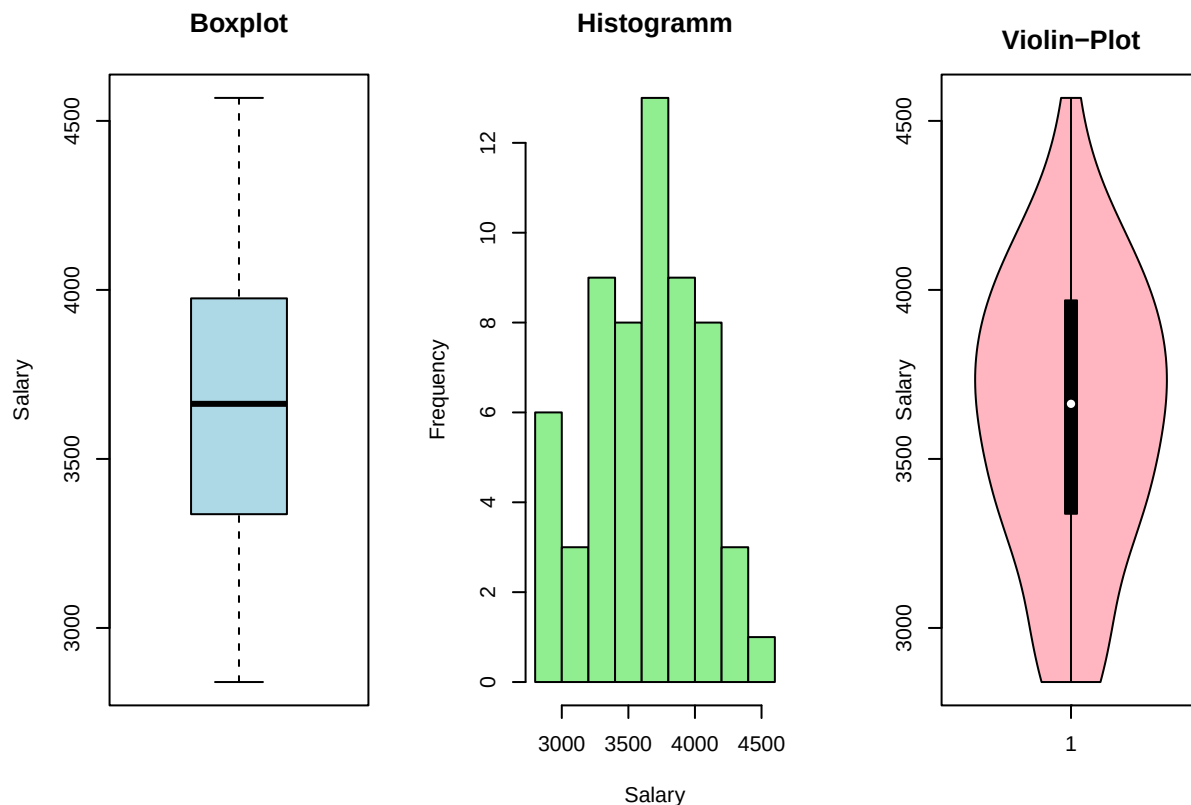
# Violin-Plot (Kombination aus Boxplot und Dichteplot)

```

```
if (!requireNamespace("vioplot", quietly = TRUE)) {
  install.packages("vioplot")
}
library(vioplot)
```

```
## Loading required package: sm
## Package 'sm', version 2.2-6.0: type help(sm) for summary information
## Loading required package: zoo
##
## Attaching package: 'zoo'
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric
```

```
vioplot(Salary,
  col = "lightpink",
  main = "Violin-Plot",
  ylab = "Salary")
```



```
# Zurücksetzen auf ein einzelnes Plot
par(mfrow = c(1, 1))
```

Statistische Erklärung: Boxplots bieten eine kompakte Darstellung der wichtigsten Verteilungseigenschaften:

1. **Box:** Repräsentiert den Interquartilsabstand (IQR) von Q1 bis Q3, der die mittleren 50% der Daten enthält.
2. **Linie in der Box:** Zeigt den Median (Q2).
3. **Whiskers (Antennen):**
 - In der Standardimplementierung in R: Erstrecken sich bis zum äussersten Datenpunkt innerhalb von $1.5 \cdot \text{IQR}$ von Q1 bzw. Q3.
 - Alternative Definitionen: Bis zum Minimum/Maximum oder bis zu bestimmten Perzentilen.
4. **Punkte ausserhalb der Whiskers:** Werden als Ausreisser betrachtet.
5. **Einkerbungen (Notches):** Wenn vorhanden, geben sie ein ungefähres 95%-Konfidenzintervall für den Median an. Überlappen sich die Einkerbungen zweier Boxplots nicht, ist dies ein Hinweis auf einen signifikanten Unterschied der Mediane.

Boxplots sind besonders nützlich für:

- Den Vergleich mehrerer Gruppen
- Die Identifikation von Ausreissern
- Die Beurteilung der Symmetrie einer Verteilung
- Die Erkennung von Unterschieden in der Streuung

R-Erklärung:

- Die Funktion `boxplot()` erstellt einen Boxplot.
- Der Parameter `notch = TRUE` fügt Einkerbungen hinzu, die ein ungefähres 95%-Konfidenzintervall für den Median darstellen.
- Der Parameter `horizontal = TRUE` erstellt einen horizontalen statt eines vertikalen Boxplots.
- Die Funktion `boxplot()` gibt eine Liste mit Statistiken zurück, die für weitere Beschriftungen verwendet werden können.
- Die Funktion `arrows()` fügt Pfeile zum Plot hinzu, die hier zur Beschriftung der Boxplot-Elemente verwendet werden.
- Das Paket `vioplot` bietet eine Alternative zum Boxplot, die zusätzlich die Dichteverteilung der Daten zeigt.

Tipp: Boxplots sind besonders nützlich zum Erkennen von Ausreissern und zum Vergleich von Gruppen. Die Box zeigt den Interquartilsabstand (IQR), der Strich in der Mitte ist der Median, und die “Whiskers” (Antennen) zeigen die Spannweite der Daten ohne Ausreisser.